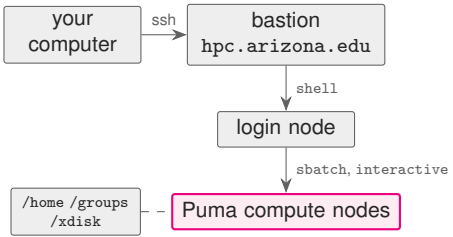
COMPUTING ON PUMA, THE UA CLUSTER

Accounts, logging in, files, the shell, and Slurm

Laura A. Miller
Department of Mathematics, University of Arizona

Using IB Software • Part I • Chapter 1



Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. get an HPC account and join your PI's group;
2. log in to Puma by SSH or through Open OnDemand;
3. move files between your computer and the cluster;
4. choose where to keep code, simulations and results;
5. work in the Linux shell: navigate, edit files, and write short scripts;
6. run work on compute nodes, interactively or as Slurm batch jobs, and check what a job used.

> Since August 2026 Puma is the University of Arizona's only HPC cluster: El Gato was shut down on 29 July 2026 and Ocelote on 17 August 2026. Older course notes for Ocelote, El Gato, and UNC's Kure, KillDevil, Dogwood and Longleaf clusters are kept in Part X, chapter X1.

HOW THE CLUSTER IS ORGANIZED

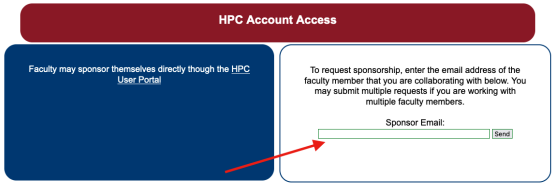


- The **bastion host** only checks your NetID, password and Duo. It stores no files and has no software.
- A **login node** is shared by everyone who is logged in. Edit files and submit jobs there.
- **Compute nodes** do the work. You reach them through **Slurm**, the scheduler, which starts your job when the cores you asked for are free.
- Never run simulations or builds on a login node. Software modules are not even available there.

GETTING AN HPC ACCOUNT

1. Create your account at https://portal.hpc.arizona.edu with your NetID.
2. If you are not faculty, ask a faculty member, your PI, to sponsor you: enter their email address on the portal's sponsorship page.
3. Your PI approves you and adds you to their **group**. The group name (for example `lauram9`) is what you put after `--account` in job scripts.

Your sponsor needs their own HPC account before they can sponsor anyone.



The sponsorship request on the HPC portal (2024)

Documentation for everything in this chapter: https://hpcdocs.hpc.arizona.edu.

WHAT YOUR GROUP GETS

Storage	Size	Use it for
`/home/...`	50 GB per person	your code, scripts, small files
`/groups/<pi>`	500 GB per group	shared, longer-term files: installs, examples
`/xdisk/<pi>`	up to 20 TB per group	running simulations and their output; lasts 150 days, with one 150-day extension

Partition	What it means
`standard`	uses your group's free monthly allocation (150,000 CPU hours on Puma, renewed on the first of each month); needs `--account=<pi group>`
`windfall`	uses no allocation, but jobs start later and can be stopped (*preempted*) when others need the cores
`high_priority`	only for groups that bought nodes in Puma

Type `va` on the cluster to see your groups and how much allocation is left.

A TERMINAL ON YOUR OWN COMPUTER

- **macOS**: open *Terminal* (Spotlight, then type “terminal”).
- **Linux**: open a terminal window.
- **Windows**: *PowerShell* and *Windows Terminal* include the `ssh` command. MobaXterm and PuTTY are graphical alternatives. Ubuntu under WSL2 (chapter 2) gives you a full Linux terminal.

A good text editor

Edit code and input files with an editor that saves plain text, never a word processor. Notepad++ and Sublime Text work well on Windows; VS Code works everywhere. On Windows, set the editor to save with Unix (LF) line endings (chapter 3 shows the error you get otherwise).

Also install

VisIt for looking at results (Part V), FileZilla for moving files (this chapter), and MATLAB or Python for IB2d (Part II).

LOGGING IN

```
ssh netid@hpc.arizona.edu
```

Enter your NetID password, then approve the Duo request. You arrive on the bastion host:

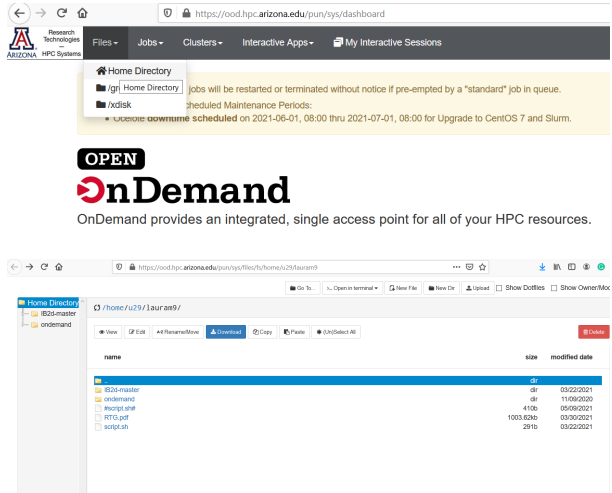
```
Success. Logging you in...
This is a bastion host used to access the rest of the RT/HPC environment.

Type "shell" to access the job submission hosts for all environments
----------------------------------------
[netid@gatekeeper ~]$ shell
```

- Typing `shell` takes you to a login node. Puma is the default cluster for job submission.
- No VPN is needed for SSH or for Open OnDemand from off campus, because Duo protects both. The HPC VPN (`vpn.hpc.arizona.edu`) is for graphical programs that cannot go through the bastion host, and is often faster for them; X11 windows also work through the bastion (`ssh -X`, then `shell -X`; chapter 31).
- Older notes say to type `puma`, `ocelote` or `elgato` here. Those were shortcuts for choosing a cluster and are no longer needed.

OPEN ONDEMAND: THE CLUSTER IN A WEB BROWSER

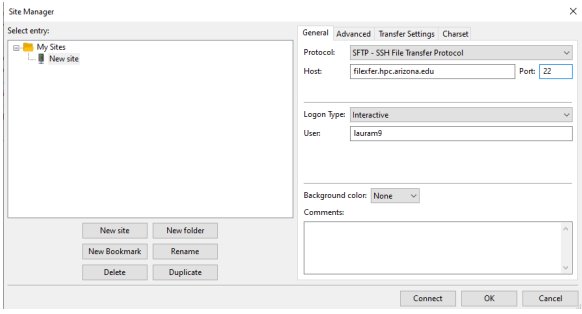
- Go to https://ood.hpc.arizona.edu and log in with your NetID and Duo.
- **Files** → **Home Directory**: browse, upload and download by drag and drop.
- **Clusters** → **Shell**: a terminal on a login node.
- **Interactive Apps**: a Linux desktop, Jupyter, MATLAB and other programs, running on compute nodes.
- Interactive apps ask for a group and partition: standard uses your allocation.

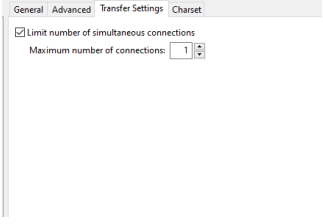


Open OnDemand's Files menu and file browser (2022; the layout has since been updated)

MOVING FILES WITH FILEZILLA

1. Install FileZilla from https://filezilla-project.org (the client, not the server).
2. **File** → **Site Manager** → **New Site**. On the **General** tab: Protocol **SFTP**, Host `filexfer.hpc.arizona.edu`, Logon Type **Interactive**, User your NetID.
3. On the **Transfer Settings** tab, tick *Limit number of simultaneous connections* and set it to **1**. Otherwise every connection asks for Duo again.
4. **Connect**. In the first password box enter your NetID password; in the second, type `1` for a Duo push or enter a Duo passcode.
5. Drag files between your computer (left) and the cluster (right).





MOVING FILES FROM A TERMINAL

sftp: an interactive session, like a small file manager:

```
sftp netid@filexfer.hpc.arizona.edu
sftp> put -r my_example          # upload a folder
sftp> get -r my_example/viz_IB2d # download a folder
sftp> exit
```

scp and **rsync**: one command, run on your own computer:

```
scp -r my_example netid@filexfer.hpc.arizona.edu:/xdisk/<pi>/<netid>/
rsync -av netid@filexfer.hpc.arizona.edu:/xdisk/<pi>/<netid>/my_example/viz_IB2d .
```

Many files at once: pack them first, then send one file.

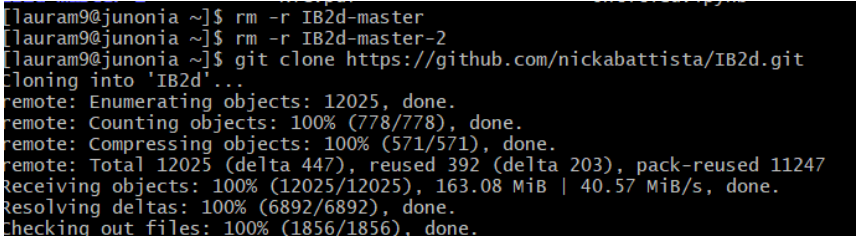
```
tar -czf my_example.tar.gz my_example      # on your computer
scp my_example.tar.gz netid@filexfer.hpc.arizona.edu:/xdisk/<pi>/<netid>/
tar -xzf my_example.tar.gz                 # on the cluster, in that folder
```

`rsync` copies only what changed and can resume an interrupted copy.

GETTING CODE WITH GIT

For code that lives on GitHub, clone it on the cluster instead of uploading it:

```
git clone https://github.com/nickabattista/IB2d.git       # first time
cd IB2d
git pull                                                   # later: get the newest version
```



Cloning IB2d on the cluster (2022)

- The tutorials and examples are on GitHub too: https://github.com/fairyflies9/Using-IB-Software.
- Keep your own simulations *outside* the cloned folder, so that `git pull` never collides with your changes.

WHERE TO KEEP WHAT

- **Home** (~, 50 GB): your scripts, small codes, a personal IBAMR install if you have one.
- **Group folder** (`/groups/<pi>`, 500 GB, shared): a group IBAMR install that everyone uses, shared examples.
- **xdisk** (`/xdisk/<pi>/<netid>`): run simulations here. Output from IB simulations is large.
- xdisk is temporary: it lasts 150 days, plus one 150-day extension, and then it is deleted. Copy results off in time.

After every run

1. Copy the results to your computer or an external drive within a week or so.
2. Check the copy.
3. Delete them from the cluster:
   `rm -r my_simulation`

A 2D run is tens of megabytes; 3D runs can reach many gigabytes. Keep results on an external drive rather than your laptop's disk.

THE SHELL: FINDING YOUR WAY AROUND

```
pwd                     # which folder am I in?
ls                      # list files
ls -lh                  # with sizes and dates
ls -a                   # include hidden files
cd my_example           # go into a folder
cd ..                   # up one level
cd ~                    # back to your home folder
mkdir runs              # make a folder
mkdir -p a/b/c          # make nested folders
cp input2d input2d.bak      # copy a file
cp -r ex1 ex1_copy          # copy a folder
mv old_name new_name        # move or rename
rm file                     # delete a file
rm -r folder                # delete a folder
```

- `rm` has no undo and no trash can. Check with `ls` before you delete.
- Press **Tab** to complete file names, and the **up arrow** to recall earlier commands.
- `man ls` shows the manual for any command; `q` quits it.
- A path starting with `/` is absolute; anything else is relative to where you are.

Adapted from the course guide “Running IBAMR/IBFE on super computers” (Nick Battista, 2014; UA version 2024).

LOOKING INSIDE FILES

```
cat input2d                    # print a whole file
less IB2d.log                  # page through it: space, b, /search, G = end, q = quit
head -n 20 input2d             # the first 20 lines
tail -n 20 IB2d.log            # the last 20 lines
tail -f IB2d.log               # keep showing new lines as a job writes them (Ctrl-C to stop)
grep "CFL number" IB2d.log     # only the lines that contain this text
grep -c "At end" run.log       # how many lines contain it
wc -l rubber_band_512.vertex # how many lines in a file
du -sh viz_IB2d                # how much disk a folder uses
```

> These are the commands you will use most when a simulation is running: `tail -f` to watch it, `grep` to pull out the numbers you care about, and `du` to keep an eye on disk use.

EDITING FILES WITH NANO

```
nano input2d
```

- Type to edit; move with the arrow keys.
- The bottom two lines list the commands; `^` means the Ctrl key.

Keys	What they do
Ctrl-O, Enter	save
Ctrl-X	quit (asks to save changes)
Ctrl-V / Ctrl-Y	next / previous page
Ctrl-W	search
Ctrl-W, then Ctrl-V	jump to the end
Ctrl-W, then Ctrl-Y	jump to the start
Ctrl-K / Ctrl-U	cut / paste a line

When you quit after changing a file, nano asks *Save modified buffer?* Type `y` to save or `n` to throw the changes away. Opening a log file only to read it? Answer `n`, or use `less` instead.

Adapted from the course guide “Running IBAMR/IBFE on super computers” (Nick Battista, 2014; UA version 2024).

EDITING FILES WITH VI

vi is on every Linux system. It has two modes:

- **Normal mode**, where you start: keys are commands.
- **Insert mode**: keys type text. Press `i` to enter it and **Esc** to leave it.

Older course notes said to press Esc to start typing. That is backwards: `i` starts typing, Esc stops.

Keys (normal mode)	What they do
`i`	start inserting text
`Esc`	back to normal mode
`:w`	save
`:wq`	save and quit
`:q`	quit (if nothing changed)
`:q!`	quit without saving
`:1215`	go to line 1215
`G` / `gg`	go to the end / the start
`/text`	search forward for *text*
`dd` / `u`	delete a line / undo

SHELL SCRIPTS: COMMANDS SAVED IN A FILE

`hello.sh`:

```
#!/bin/bash
# The first line says which program runs the
    file.
echo "Hello from $(hostname)"
echo "I am in $PWD"
```

Run it:

```
bash hello.sh
# or make it executable once, then:
chmod +x hello.sh
./hello.sh
```

- A script runs its commands one after another, exactly as if you typed them.
- # starts a comment, except in the first line, `#!/bin/bash`, which names the shell.
- Slurm job scripts (later in this chapter) are shell scripts with extra `#SBATCH` comment lines that Slurm reads.
- `bash`, the “Bourne-again shell”, is the standard shell on Linux and on Puma; older shells such as `sh` and `csh` differ in small ways.

VARIABLES AND QUOTING

```
MU=10                       # no spaces around =
echo $MU                    # => 10
echo "MU is $MU"            # => MU is 10   (double quotes expand variables)
echo 'MU is $MU'            # => MU is $MU  (single quotes do not)
RUN=run_${MU}               # braces mark where the name ends => run_10
NSTEPS=$(grep -c "At end" run.log)    # $( ) captures a command's output
echo "finished $NSTEPS steps"
```

- `MU = 10`, with spaces, is an error: the shell tries to run a command called `MU`.
- Put double quotes around variables that hold file names, so names with spaces survive: `cp "$FILE" backup/`.
- `$HOME`, `$USER` and `$PWD` are always set; Slurm adds variables such as `$SLURM_NTASKS` inside jobs.

Shell examples in this chapter rewritten from the 2018 course deck “Running many simulations, part 1”, which drew on Learn X in Y minutes (bash), Machtelt Garrels’s *Bash Guide for Beginners* (the deck called it the Rutgers Bash Beginners Guide), and DigitalOcean’s awk tutorial.

LOOPS AND TESTS

```
# make three run folders
for MU in 100 10 1; do
    mkdir -p run_MU$MU
    echo "made run_MU$MU"
done

# numbers 1 to 5
for i in {1..5}; do echo $i; done
for ((i=1; i<=5; i++)); do echo $i; done
```

```
if [ -f input2d ]; then
    echo "input2d is here"
else
    echo "no input2d"
fi

# run the second command only if the first
    worked
cmake --build build && echo "build OK"
# ... only if it failed
cmake --build build || echo "build FAILED"
```

- Inside `[ ]` the spaces are required: `[ -f input2d ]`.
- Useful tests: `-f` a file exists, `-d` a folder exists, `"$a" == "$b"` equal strings, `$x -lt 5` a number less than 5.

REDIRECTION AND PIPES

```
mpirun -np 4 ./build/main2d input2d > run.log          # screen output into a file
mpirun -np 4 ./build/main2d input2d > run.log 2>&1     # ... errors too
echo "MU = 10" >> notes.txt                            # append instead of overwrite
grep "integral{F}" run.log | tail -1                    # the last force line
grep "CFL number" IB2d.log | awk '{print $NF}' > cfl.txt   # just the numbers
```

- Every program has three streams: **input** (0), **output** (1) and **errors** (2). `>` sends output to a file, `2>&1` sends errors to the same place.
- `|` (a *pipe*) sends one command's output into the next command as its input. Chains of small commands do a lot of everyday data handling.
- `> file` overwrites `file` without asking.

THREE TOOLS FOR TEXT: GREP, SED, AWK

```
grep -n "MU" input2d                      # lines containing MU, with line numbers
grep -r "viz_dump_interval" .             # search every file below this folder
sed -n '10,20p' input2d                   # print lines 10 to 20
sed -i 's/^END_TIME .*/END_TIME = 0.5/' input2d    # replace the END_TIME line (edits the file!)
awk '{print $1, $2}' F.00400              # print columns 1 and 2
awk '/^MU[ \t]*=/ {print "MU = 10"; next} {print}' input2d > input2d.new   # replace by name
awk -F, '{s += $3} END {print s/NR}' data.csv      # average of column 3 in a CSV file
```

- `grep` finds lines, `sed` edits lines, `awk` works with columns and does arithmetic.
- `sed -i` changes the file in place: make a copy first (`cp input2d input2d.bak`).
- awk splits each line into fields `$1`, `$2`, ... at spaces; `-F,` splits at commas. `$NF` is the last field, `NR` the line number. A `BEGIN` or `END` block runs before or after all lines.

SOFTWARE ON PUMA: MODULES

Software is installed as **modules**, which you load on a compute node:

```
interactive                  # a compute node: 1 core, 1 hour, windfall
module avail                 # everything that is installed
module avail matlab          # just MATLAB versions
module load gnu13 openmpi5   # the compilers and MPI used for IBAMR in this course
module list                  # what is loaded now
module unload openmpi5       # remove one
```

What `interactive` does (Puma, 2024):

```
Run "interactive -h for help customizing interactive use"
Submitting with /usr/local/bin/salloc --job-name=interactive --mem-per-cpu=4GB --nodes=1 --ntasks=1 --time=01:00:00
      --account=windfall --partition=windfall
NOTICE: Submitting to the windfall queue may result in longer wait times or preemption. Consider submitting with an account (-a
      your_group).
salloc: Granted job allocation 10095845
```

`interactive -a <your group>` uses your group's allocation and usually starts sooner;
`interactive -h` lists the options for more cores or time.

A SLURM BATCH SCRIPT

```
#!/bin/bash
### Part 1: what to reserve
#SBATCH --job-name=hello_world
#SBATCH --output=%x-%j.out        # %x = job name, %j = job number
#SBATCH --account=<PI GROUP>      # required
#SBATCH --partition=standard      # or windfall
#SBATCH --nodes=1
#SBATCH --ntasks=1                # cores (MPI processes)
#SBATCH --mem-per-cpu=4gb
#SBATCH --time=00:10:00           # hh:mm:ss
### #SBATCH --mail-type=ALL       # optional: email at start and end
### #SBATCH --mail-user=<netid>@arizona.edu

### Part 2: what to run
module load gnu13
cd "$SLURM_SUBMIT_DIR"
./hello_world
```

- Lines starting `#SBATCH` are read by Slurm. To switch one off, add more `#`: `### #SBATCH ...`
- **Part 1** reserves resources: cores, memory, time, group, partition.
- **Part 2** is an ordinary shell script that runs on the compute node.
- Ask for a little more time than you need. A job that runs out of time is stopped.
- Every example in this tutorial has a ready `job.slurm` (chapter 3).

Based on the UArizona HPC quick-start template, as used in the course decks (2022); updated module and memory lines.

SUBMITTING, WATCHING AND CHECKING JOBS

```
sbatch job.slurm                  # submit: "Submitted batch job 10095996"
squeue --me                       # your jobs: ST = PD (pending) or R (running)
scancel 10095996                  # stop a job
sacct -j 10095996 --format=JobID,State,Elapsed,AllocCPUS,MaxRSS,ExitCode   # afterwards
```

- `squeue --me` is the same as `squeue -u <netid>`. The `NODELIST(REASON)` column says why a job is waiting, for example `(Priority)`.
- `sacct` reports what a finished job used: `Elapsed` time and `MaxRSS`, the most memory one process used. Use them to size the next request.
- Write down each job number with a note of what it runs; chapter 3 shows how to follow an IBAMR job.

`squeue --me` and `sacct` usage from the instructor's 2026 Puma notes for IB2d runs.

MATLAB AND PYTHON JOBS (FOR IB2D)

```
#!/bin/bash
#SBATCH --job-name=ib2d_run
#SBATCH --account=<PI GROUP>
#SBATCH --partition=standard
#SBATCH --ntasks=1
#SBATCH --mem=16gb
#SBATCH --time=04:00:00

module load matlab
cd "$SLURM_SUBMIT_DIR"
matlab -nodisplay -nosplash < main2d.m
```

- IB2d runs on one core, so `--ntasks=1`; ask for enough memory for the grid you use.
- For the Python version of IB2d, load a Python module instead, for example `module load python/3.11/3.11.4`, and run `python main2d.py`. Module names change; check with `module avail python`.
- If MATLAB asks for a MathWorks sign-in, start an `interactive` session, load the same module, run `matlab` once and sign in. Do not do this on a login node.
- Part II covers IB2d itself.

Adapted from the course decks “HPC @ UA, running ib2d and IBAMR” (2021–2022) and the instructor’s 2026 Puma notes.

OTHER UNIVERSITY COMPUTING SERVICES

These are useful but not needed for this tutorial:

- **U-System**: a general-purpose Unix account (https://account.arizona.edu), reached by SSH to `u.arizona.edu`.
- **Virtual desktops** (VCAT, https://arizona.apporto.com): Windows desktops in a browser with licensed software such as ANSYS, MATLAB, Blender and MeshLab, without installing anything. They can be slow when many people use them.
- **Jupyter and MATLAB in a browser**: through Open OnDemand's Interactive Apps on Puma.

> Check what each service offers on the University's IT pages before relying on it; the lists of software change every year.

From the course deck "HPC and computing resources @ UA" (2022).

WHAT CHANGED SINCE THE OLDER COURSE NOTES

Older notes say	Now
log in to Puma, Ocelote or El Gato; type `puma` or `ocelote`	Puma only; type `shell` at the bastion
UNC clusters: Kure, KillDevil, Dogwood, Longleaf (LSF `bsub`, then Slurm)	see Part X, chapter X1
start a VPN before logging in	not needed for SSH or Open OnDemand
documentation on Confluence (`public.confluence.arizona.edu`)	https://hpcdocs.hpc.arizona.edu
`module load gnu/5.4.0`	`module load gnu13 openmpi5` for IBAMR
xdisk is deleted after 300 days	150 days plus one 150-day extension: 300 at most
`#SBATCG --output` (a typo, so the line was ignored)	`#SBATCH --output`
`-t 1` means one hour	`-t 1` means one *minute*; write `--time=01:00:00`
in vi, press Esc to type	press `i` to type, Esc to stop

SUMMARY

1. Get an account on the HPC portal and join your PI's group; the group name goes after `--account`.
2. Log in with `ssh netid@hpc.arizona.edu`, approve Duo, type `shell`. Or use Open OnDemand.
3. Move files with FileZilla, `sftp`, `scp` or `rsync` through `filexfer.hpc.arizona.edu`; get code with `git clone`.
4. Code in home or the group folder; simulations on xdisk; copy results off and delete them.
5. Learn the shell: `ls`, `cd`, `less`, `grep`, an editor, and short scripts.
6. Work on compute nodes only: `interactive` for short tests, `sbatch` for real runs, `squeue --me` and `sacct` to follow them.

Next, chapter 2 installs IBAMR, and chapter 3 builds and runs your first simulation.

REFERENCES AND CREDITS

- UArizona HPC documentation: https://hpcdocs.hpc.arizona.edu (system access, storage, allocations, Open OnDemand, file transfer).
- FileZilla: https://filezilla-project.org.
- Course material reused here: “Running IBAMR/IBFE on super computers” (Nick Battista, 2014; UA version 2024); “HPC and computing resources @ UA” and “HPC @ UA, running ib2d and IBAMR” (2021–2022); “Running many simulations, part 1” (2018); the instructor’s Puma notes (2026).
- Shell references the 2018 deck drew on: Learn X in Y minutes, bash (https://learnxinyminutes.com/docs/bash/); Machtelt Garrels, *Bash Guide for Beginners*; DigitalOcean, “How to use the awk language to manipulate text in Linux”.
- Screenshots are from the course decks (2022–2024).